BombLab 实验报告

环境：WSL2 Ubuntu / x86-64 Linux；仓库 ~/ics-2026-bomblab。使用 GCC、Make、objdump、GDB 和 nano。

1 第 1 关：维护站

1.1 输入检查与调用关系

```
40209d: mov  $0x1,%esi
4020aa: call valid_u32_fields
4020ce: call __isoc99_sscanf
4020d3: cmp  $0x1,%eax
4020d8: mov  %r12,%rdi
4020db: call p1_compute
4020e0: cmp  %eax,-0x14(%rbp)
4020e3: sete %al
```

字段检查的参数为 1，sscanf 的返回值也与 1 比较，因此输入只有一个整数。末尾的 cmp 将它与 p1_compute 的返回值比较，计算过程需要继续追进 p1_compute。

p1_compute 依次调用三个 step。将 challenge 的低 32 位、高 32 位和右移 16 位后的低 32 位记为 lo、hi、mid，调用分别为 p1_step0(lo,hi)、p1_step1(hi,mid)、p1_step2(s1,lo)。step0 的结果暂存 ecx，最后与 step2 的结果异或。

1.2 计算关系与实现

函数	关键指令	运算（a、b 为两个参数）
p1_step0	lea 0xf39b(%rsi),%eax xor %edi,%eax	(b + 0xf39b) ^ a
p1_step1	xor $0x5727,%esi lea (%rsi,%rdi,8),%eax	(b ^ 0x5727) + 8*a
p1_step2	sub %esi,%edi lea 0x55d1(%rdi,%rdi,4),%eax	5*(a-b) + 0x55d1

```
uint32_t lo  = (uint32_t)challenge;
uint32_t hi  = (uint32_t)(challenge >> 32);
uint32_t mid = (uint32_t)(challenge >> 16);
uint32_t s0 = (hi + 0xF39Bu) ^ lo;
uint32_t s1 = (mid ^ 0x5727u) + 8u * hi;
uint32_t s2 = 5u * (s1 - lo) + 0x55D1u;
values[0] = s2 ^ s0;
count = 1;
```

右移在 64 位 challenge 上进行，结果再转为 uint32_t。三个 step 的运算保留低 32 位，最终异或值写入 values[0]，作为唯一的输出字段。

1.3 第 1 关验证记录

验证输入	结果	证据
公开列表的 10 个 seed	Public tests: 10/10	图 1
seed 43（公开列表外）	Phase 1 defused.	图 2

```
./defuser --seed 43 --phase 1 | ./bomb --seed 43 --phase 1
```

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ python3 public_test.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 1 第 1 关公开测试：10 个 seed 全部 PASS。

```
=== seed=43 ===

Phase 1 challenge: 0x9cde98852e60034b
Input (unsigned decimal integers):
Phase 1 defused.
```

图 2 非公开 seed 43 的运行结果。

2 第 2 关：钟楼

2.1 六个输入字段与第一项

```
4020fa: mov  %rsi,%r13
4020fd: mov  $0x6,%esi
40210c: call valid_u32_fields
402132: mov  $0x40317a,%esi
40213c: lea  -0x40(%rbp),%rdx
40214e: call __isoc99_sscanf
402155: cmp  $0x6,%eax
```

入口的 challenge 保存到 r13。GDB 用 x/s 0x40317a 读出格式串 "%u %u %u %u %u %u"；调用后 eax=6。六个 uint32_t 连续存放在 rbp-0x40 至 rbp-0x2c，每项间隔 4 字节。

```
40215a: mov    %r13,%rax
40215d: mov    -0x40(%rbp),%esi
402160: shr    $0x2a,%rax
402164: movzbl %al,%eax
402167: add    $0x8,%eax
40216a: cmp    %eax,%esi
40216c: jne    402115
```

第一项要求值由 challenge 右移 42 位、取低 8 位、加 8 得到。esi 保存第一个输入，与 eax 中的要求值不相等时跳到失败出口。

2.2 循环与递推关系

```
40216e: mov  $0x1,%edx
402173: lea  0x4(%rdx),%ecx
402176: mov  %r13,%rax
402179: shr  %cl,%rax
40217c: lea  0xa(%rdx),%ecx
40217f: and  $0xf,%eax
402182: xor  %ecx,%eax
402184: lea  (%rsi,%rsi,4),%ecx
402187: mov  -0x40(%rbp,%rdx,4),%esi
40218b: add  %ecx,%eax
40218d: cmp  %eax,%esi
40218f: jne  402115
402191: add  $0x1,%rdx
402195: cmp  $0x6,%rdx
402199: jne  402173
40219b: jmp  402117
```

将 rdx 记为下标 i：它从 1 开始，每轮加 1，到 6 时退出，因此检查 a[1] 至 a[5]。每轮重新从 r13 取 challenge，右移 i+4 位，取低 4 位，再与 i+10 异或。

402184 的 lea 计算 rsi+4*rsi。此时 esi 仍是上一项；402187 才从数组读入当前项。因此该乘加使用前一项的 5 倍。要求值与当前项比较通过后，当前项留在 esi，供下一轮使用。

所有比较通过后，40219b 跳到 402117 的公共收尾，避开 402115 清零返回值的失败路径。

```
values[0] = (uint32_t)((challenge >> 42) & 0xFFu) + 8u;
for (uint32_t i = 1; i < 6; ++i) {
    uint32_t part = (uint32_t)((challenge >> (i + 4u)) & 0xFu);
    values[i] = 5u * values[i - 1] + (part ^ (i + 10u));
}
count = 6;
```

六项按下标顺序写入 values。challenge 保持 64 位无符号类型，数组元素和每轮附加值使用 uint32_t。

2.3 GDB 观察与推理核对

我用 seed 42 调试第二关，challenge 为 0x47526757130f9f52。输入六个 0 后，确认 sscanf 读入了六项以及数组的位置。第一项算出是 161，于是改用 161 0 0 0 0 0 重新运行，观察后面的循环。

观察位置 / 操作	记录与含义
sscanf 返回后	eax=6；x/6uw $rbp-0x40 显示六个 0
第一项：右移 42 位	rax=0x11d499
取低 8 位，再加 8	0x99=153；加 8 后要求值为 161
首轮循环，i=1	ecx=5；challenge 右移 5 位后为 0x23a933ab8987cfa
首轮附加值及递推核算	低 4 位为 10；10 ^ 11=1；5*161+1=806

2.4 第 2 关验证记录

验证输入	结果	证据
seed 42	Phase 2 defused.	图 3
公开列表的 10 个 seed	Public tests: 10/10	图 4
seed 43（公开列表外）	Phase 2 defused.	图 5

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ ./defuser --seed 42 --phase 2 | ./bomb --seed 42 --phase 2
Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
Phase 2 defused.
```

图 3 seed 42 的单关命令与通过结果。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 4 第 2 关公开测试：10 个 seed 全部 PASS。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 2 | ./bomb --seed 43 --phase 2
Phase 2 challenge: 0x6ec624aa85685867
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

图 5 非公开 seed 43：命令、challenge 与 Phase 2 defused.

3 第 3 关：分流站

3.1 输入条件与分支参数

```
4022ad: mov  $0x2,%esi
4022ba: call valid_u32_fields
4022e7: cmp  $0x2,%eax
4022f1: and  $0x7,%edi
4022f4: cmp  %edi,-0x18(%rbp)
4022ff: shr  $0x3,%rax
402303: shr  $0x20,%rsi
402307: xor  %eax,%esi
402309: call p3_dispatch
40230e: cmp  %eax,-0x14(%rbp)
402311: sete %al
```

入口将 challenge 保存到 r12。字段校验要求 2 个无符号整数；sscanf 的两个输出指针分别指向 rbp-0x18 与 rbp-0x14。第一项必须等于 challenge 的低 3 位，范围为 0-7。

```
index = (uint32_t)challenge & 7;
x = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3);
```

右移在 64 位 challenge 上执行，异或使用两个结果的低 32 位。调用前 edi 为 index、esi 为 x。两个答案依次为 index 和 p3_dispatch(index,x) 的返回值；末尾 cmp 与 sete 要求第二项等于 eax。

3.2 跳转表对应的分支

```
402244: mov  %edi,%eax
402246: mov  %esi,%edi
402248: cmp  $0x7,%eax
40224b: ja   402295
40224d: notrack jmp *0x405820(,%rax,8)
```

分发函数将编号保存到 eax，再把 x 移入 edi，作为 case 函数的第一个参数。ja 对无符号编号做范围检查；间接跳转读取地址 0x405820+8*index 处的 8 字节代码地址。

GDB 执行 x/8gx 0x405820 得到下表。表项先跳到一条 jmp，再进入实际 case 函数，因此需按表项内容确定对应关系。

编号	跳转落点	最终函数	恢复出的返回值
0	0x402268	p3_case_2	3*x + 0xf344
1	0x402270	p3_case_6	(x ^ 0xcdf3) + (x >> 1)
2	0x402278	p3_case_1	5*x + 0x49fd4
3	0x402280	p3_case_7	3*x + 0x932e
4	0x402288	p3_case_3	(2*x) ^ 0x7519
5	0x402290	p3_case_5	5*x + 0x4d25b
6	0x402258	p3_case_0	(x ^ 0x40b5) + (x >> 2)
7	0x402260	p3_case_4	(4*x) ^ 0xec2b

3.3 分支公式与代码实现

进入 case 函数时，edi 中的参数就是 x。lea 按表达式计算，shr 做逻辑右移；写入 eax 的结果只保留低 32 位。

```
p3_case_2:
  4021d4: lea 0xf344(%rdi,%rdi,2),%eax
p3_case_6:
  402224: mov %edi,%eax
  402226: shr $1,%edi
  402228: xor $0xcdf3,%eax
  40222d: add %edi,%eax
p3_case_3:
  4021e4: lea (%rdi,%rdi,1),%eax
  4021e7: xor $0x7519,%eax
```

case_2 的 lea 得到 x+2*x+0xf344。case_6 把原始 x 留在 eax，右移 edi 中的副本，因此返回值是 (x^0xcdf3)+(x>>1)。在 switch(index) 中，表下标 1 对应的代码为：

```
case 1:
    values[1] = (x ^ 0xCDF3u) + (x >> 1);
    break;
```

其余分支按表中的公式填写。第一项写入 values[0]=index，第二项写入分支计算结果，count 设为 2。

3.4 第 3 关验证记录

各分支公式填入后，我运行第三关的公开测试，10 个 seed 全部通过（图 6）。用公开列表外的 seed 43 做单关验证，bomb 输出 Phase 3 defused.（图 7）。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 6 第 3 关公开测试：10 个 seed 全部 PASS。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 3 | ./bomb --seed 43 --phase 3
Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.
```

图 7 非公开 seed 43 的单关命令与通过结果。

seed 43 的 challenge 为 0xd49ffe504bfdab7f，得到 index=7、x=0xdde04b3f。按表进入 case_4，代入可得输出 7 2004992215。

4 第 4 关：档案馆

4.1 输入条件与递归参数

```
40239a: mov  %rsi,%r12
40239d: mov  $0x1,%esi
4023aa: call valid_u32_fields
4023d3: cmp  $0x1,%eax
4023e1: xor  %esi,%esi
4023e3: shr  $0x20,%rax
4023e7: shr  $0x7,%rcx
4023eb: and  $0x7f,%edi
4023ee: mov  $0x7f,%edx
4023f3: xor  %eax,%ecx
4023f5: call p4_walk
4023fa: cmp  %eax,-0x14(%rbp)
4023fd: sete %al
```

输入仍为一个整数，存放在 rbp-0x14，与 p4_walk 的返回值比较。调用前的赋值给出了 target、key 及初始区间；在求解器中由 walk_phase4 实现这段递归。

```
uint32_t target = (uint32_t)challenge & 0x7Fu;
uint32_t key = (uint32_t)(challenge >> 7) ^ (uint32_t)(challenge >> 32);
values[0] = walk_phase4(target, 0, 127, key);
count = 1;
```

调用时 edi、esi、edx、ecx 分别为 target、low=0、high=127、key。两次右移作用于 64 位 challenge，再按低 32 位异或；target 取最低 7 位，范围为 0-127。

4.2 区间中点、结束条件与递归方向

```
402324: mov  %edi,%eax
402326: mov  %edx,%edi
402328: sub  %esi,%edi
40232a: shr  $1,%edi
40232c: add  %esi,%edi
40232e: cmp  %eax,%edi
402330: je   402380
40233e: cmp  %edi,%eax
402340: jb   402360
```

target 暂存于 eax，edi 改为中点 mid=low+((high-low)>>1)。相等时跳到基例；否则 cmp %edi,%eax 比较 target 与 mid，jb 表示无符号 target<mid。

条件	参数变化的关键指令	下一步
target == mid	je 402380	直接计算基例返回值
target < mid	402360: lea -0x1(%rdi),%edx 402363: mov %eax,%edi	p4_walk(target, low, mid-1, key)
target > mid	402342: lea 0x1(%rdi),%esi 402345: mov %eax,%edi	p4_walk(target, mid+1, high, key)

target 在初始区间 [0,127] 内。每次调用保留它所在的半区，key 不变，直到 mid==target。子调用返回后，当前这一层还有运算要完成。

4.3 递归返回值与实现

402337 的 lea (%rdi,%rcx,1),%ebx 保存本层的 mid+key。子调用返回后，eax 中的结果乘 5，再与本层保存值和分支常数异或。将子调用返回值记为 r，可得：

情况	关键计算指令	返回值
找到目标	lea 0xb2(%rdi),%eax xor %ecx,%eax	(mid + 0xb2) ^ key
向左递归后返回	lea (%rax,%rax,4),%eax xor %ebx,%eax; xor $0x1c,%eax	(5*r) ^ (mid + key) ^ 0x1c
向右递归后返回	lea (%rax,%rax,4),%eax xor %ebx,%eax; xor $0x3,%eax	(5*r) ^ (mid + key) ^ 0x03

实现时保留汇编中的递归结构：找到 target 时直接返回，另外两个分支取得子调用结果后，继续计算本层返回值。walk_phase4 的主体为：

```
uint32_t mid = low + ((high - low) >> 1);
if (target == mid) return (mid + 0xB2u) ^ key;
if (target < mid) {
    uint32_t r = walk_phase4(target, low, mid - 1u, key);
    return (5u * r) ^ (mid + key) ^ 0x1Cu;
}
uint32_t r = walk_phase4(target, mid + 1u, high, key);
return (5u * r) ^ (mid + key) ^ 0x03u;
```

4.4 第 4 关验证记录

编译通过后，用公开测试检查递归函数，结果为 10/10（图 8）。seed 43 的单关管道运行也通过（图 9）。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ make && python3 public_test.py --phase 4
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 8 第 4 关公开测试命令与 10/10 PASS 结果。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 4 | ./bomb --seed 43 --phase 4
Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.
```

图 9 非公开 seed 43：单关命令与 Phase 4 defused.

seed 43 的 challenge 为 0x006ff81b72d591ff，得到 target=127、key=0x368a5338。代入递归计算，口令为 403073518。

5 第 5 关：巡检回廊

5.1 两个输入与两种结果传递方式

```
40247d: mov  $0x2,%esi
40248a: call valid_u32_fields
4024bc: lea  -0x14(%rbp),%rsi
4024c0: mov  %r12,%rdi
4024c3: call p5_traverse
4024c8: mov  %eax,%edx
4024cc: cmp  %edx,-0x1c(%rbp)
4024d1: mov  -0x14(%rbp),%eax
4024d4: cmp  %eax,-0x18(%rbp)
```

本关读取 2 个整数，分别放在 rbp-0x1c 与 rbp-0x18。p5_traverse 的第一个参数是 challenge，第二个参数是 rbp-0x14 的地址。函数返回的累积值检查第一项；写入该地址的末节点标签检查第二项。

5.2 节点布局与链表连接

add (%rdx),%eax 读取偏移 +0 的 4 字节 value；mov 0x4(%rdx),%esi 读取偏移 +4 的 4 字节 tag；mov 0x8(%rdx),%rdx 读取偏移 +8 的 8 字节 next 指针。每个节点占 16 字节。

```
x/8gx 0x405b60
x/48wx 0x405ba0
```

GDB 读出的头表与 12 个节点如下。节点编号按起点 0x405ba0、步长 0x10 分配；next 一列把实际地址换成对应节点编号。

编号	节点地址	value	tag	next 编号
0	0x405ba0	0xce7b	0x0042	11
1	0x405bb0	0x4ee0	0x0654	7
2	0x405bc0	0xc579	0x0b4a	6
3	0x405bd0	0x3f4a	0x0a6c	8
4	0x405be0	0x28a2	0x0417	10
5	0x405bf0	0xd6c0	0x0960	3
6	0x405c00	0x439e	0x0782	9
7	0x405c10	0x2ef9	0x022c	4
8	0x405c20	0x5789	0x0541	2
9	0x405c30	0x55ba	0x013b	1
10	0x405c40	0x4c2c	0x0887	0
11	0x405c50	0x432d	0x035b	NULL

头表换算为 heads[0..7] = [5, 3, 8, 2, 6, 9, 1, 7]。代码中用数组保存 value、tag，用节点编号代替 next 地址，仍按原来的连接顺序访问。

5.3 由 challenge 选择起点与初始值

```
40241d: and  $0x7,%eax
402420: shr  $0x3,%rcx
402424: mov  0x405b60(,%rax,8),%rdx
40242f: shr  $0x20,%rdi; 402433: shr $0x6,%rax
402437: xor  %edi,%eax
40243e: xor  $0xd4ec,%eax
```

函数先把 challenge 复制到 rax、rcx，得到 node=heads[challenge & 7]；初始 acc=low32(challenge>>6) ^ low32(challenge>>32) ^ 0xd4ec。两次右移在 64 位寄存器中进行，随后按低 32 位异或。

5.4 访问次数与遍历实现

```
402443: add  (%rdx),%eax
402445: lea  (%rax,%rax,4),%eax
402448: mov  %esi,(%r8)
40244b: xor  %esi,%eax
40244d: and  $0x3,%edi
402450: je   40246f
402457: mov  0x8(%rdx),%rdx
40246b: cmp  %ecx,%edi
40246d: jae  402457
```

头节点的运算发生在循环之前。令 t=(challenge>>3)&3，t=0 时处理完头节点就返回，其余情况沿 next 再前进 t 次。因此代码中的访问次数为 t+1：

```
unsigned visits = (unsigned)((challenge >> 3) & 3u) + 1u;
uint32_t last_tag = 0;
for (unsigned i = 0; i < visits; ++i) {
    last_tag = node_tags[node];
    acc = (5u * (acc + node_values[node])) ^ last_tag;
    if (i + 1u < visits) node = next_node[node];
}
values[0] = acc;
values[1] = last_tag;
```

acc 使用 uint32_t，每轮加上 value、乘 5、再与 tag 异或。last_tag 随访问覆盖，结束时正好保存最后一个节点的标签；两个值分别写入 values[0] 和 values[1]。

5.5 第 5 关验证记录

我把节点数据和遍历循环写入求解器后，重新编译并运行第五关测试。公开 seed 全部通过（图 10），seed 43 也显示 Phase 5 defused.（图 11）。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ make && python3 public_test.py --phase 5
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 10 第 5 关公开测试命令与 10/10 PASS 结果。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 5 | ./bomb --seed 43 --phase 5
Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.
```

图 11 非公开 seed 43：单关命令与通过结果。

seed 43 的 challenge 为 0x18e47d0995aecbfe。由公式推演，头表编号为 6、访问 4 个节点，路径为 1→7→4→10，计算得到 acc=295688137、末标签为 0x887（2183），输出为 295688137 2183。

6 第 6 关：交接室

phase_06 将输入与 p6_dispatch(challenge) 的返回值比较。p6_dispatch 中有两次通过函数表发起的调用，参数和下标需要分别追踪。

6.1 输入位置与通过条件

```
4025ca: mov  %rsi,%r12
4025cd: mov  $0x1,%esi
4025da: call valid_u32_fields
4025f2: lea  -0x14(%rbp),%rdx
4025f6: mov  $0x403189,%esi
4025fe: call __isoc99_sscanf
402603: cmp  $0x1,%eax
402608: mov  %r12,%rdi
40260b: call p6_dispatch
402610: cmp  %eax,-0x14(%rbp)
402613: sete %al
```

入口的第二个参数 challenge 保存在 r12；字段检查要求 1 项。格式地址 0x403189 的内容为 "%u"，sscanf 将整数写入 rbp-0x14，并检查成功赋值数量为 1；失败时返回 0。调用 p6_dispatch 后，cmp 比较其 eax 返回值与输入，sete 在相等时把 al 设为 1。因此目标是求出 p6_dispatch(challenge)。

6.2 沿参数与返回值追踪两次间接调用

```
402579: mov  %rdi,%rbx
40257c: shr  $0x6,%rdi
402580: mov  %rbx,%rax
402583: shr  $0x20,%rax
402587: xor  %eax,%edi
402589: mov  %rbx,%rax
402590: and  $0x38,%ebx
402593: and  $0x7,%eax
402596: call *0x405e00(,%rax,8)
40259d: xor  $0xe39d,%eax
4025a2: mov  %eax,%edi
4025a4: call *0x405e00(%rbx)
4025aa: mov  -0x8(%rbp),%rbx
4025ae: leave
4025af: add  $0xa29d,%eax
4025b4: ret
```

令 c=challenge。rbx 先保存原始 c；rdi 右移 6 位，rax 取 c 右移 32 位，随后通过 edi、eax 做 32 位异或。第一次调用的参数与两个选择编号可恢复为：

```
x = (uint32_t)(c >> 6) ^ (uint32_t)(c >> 32);
i = c & 7;
j = (c >> 3) & 7;
```

第一条 call 读取地址 0x405e00+8*i 处的函数指针。第二条 call 使用 rbx=c&0x38 作为字节偏移：0x38 保留第 3-5 位，偏移只能是 0、8、…、56；每项 8 字节，故下标 j=(c&0x38)/8。第一次返回值先异或 0xe39d，再由 eax 复制到 edi，成为第二次的参数。

push/pop 保存与恢复寄存器，栈空间分配用于调用准备；leave 恢复栈帧，ret 才返回。因此 leave 后的 add 仍属于本关算法，最终结果还要加 0xa29d。

6.3 函数表与运算编号

用 x/8gx 0x405e00 按 8 字节读取函数表。每列对应一个下标，列内给出目标地址和函数名。

0	1	2	3	4	5	6	7
0x4024f0 p6_op_1	0x402560 p6_op_5	0x402560 p6_op_5	0x402520 p6_op_3	0x402540 p6_op_4	0x402500 p6_op_2	0x4024e0 p6_op_0	0x4024f0 p6_op_1

函数表对应 ops={1,5,5,3,4,2,0,1}，据此把表下标转换为实际运算编号。

6.4 逐个恢复六种 uint32_t 运算

每个运算入口的 edi 为 x，eax 为返回值。跟踪复制后的两个寄存器，可以区分原始 x 与已经修改的中间值。

函数	关键汇编（省略 endbr64、ret）	逐步翻译
op_0	lea 0x5081(%rdi),%eax shr $7,%edi xor %edi,%eax	a=x+0x5081；b=x>>7 返回 a^b；右移使用原始 x
op_1	mov %edi,%eax rol $5,%eax xor $0xc44c,%eax	a=rol32(x,5) 返回 a^0xc44c
op_2	mov %edi,%eax shl $7,%edi xor $0xb7f1,%eax add %edi,%eax	a=x^0xb7f1；b=x<<7 返回 a+b
op_3	lea 0xf464(,%rdi,4),%eax lea (%rdi,%rdi,2),%edx xor %edx,%eax	a=4*x+0xf464；b=x+2*x 返回 a^b，即 (4*x+0xf464)^(3*x)
op_4	mov %edi,%eax shr $2,%eax xor %edi,%eax add $0xeb,%eax	a=(x>>2)^x 返回 a+0xeb
op_5	mov %edi,%eax ror $5,%eax	返回 ror32(x,5)

```
rol32(x, 5) = (x << 5) | (x >> 27);
ror32(x, 5) = (x >> 5) | (x << 27);
```

rol、ror 会把移出的位接回另一端，故用两个移位结果按位或。lea 按地址表达式计算，不访问该表达式指向的内存。各运算及末尾加法均按 uint32_t 截断，右移为逻辑右移；初始 c 先做 64 位右移，再取低 32 位。

6.5 组合关系并落实到 defuser.c

代码中用 phase6_op(op,x) 实现六种运算，solve_phase_6 按下标调用两次。两次调用之间异或 0xe39d，最后加 0xa29d，将结果写入唯一的输出字段。

```
static const uint32_t ops[8] = {1, 5, 5, 3, 4, 2, 0, 1};
uint32_t i = (uint32_t)challenge & 7u;
uint32_t j = (uint32_t)(challenge >> 3) & 7u;
uint32_t x = (uint32_t)(challenge >> 6)
           ^ (uint32_t)(challenge >> 32);
x = phase6_op(ops[i], x);
x ^= 0xE39Du;
x = phase6_op(ops[j], x);
values[0] = x + 0xA29Du;
count = 1;
```

6.6 seed 43 验算与测试

公开列表外的 seed 43 对应 challenge=0x523fb4dd5d162e88，代入前述公式得到：

步骤	按恢复公式验算的结果
提取两次下标	i=0，j=1，因此先执行 op_1，再执行 op_5
计算初始 x	low32(c>>6) ^ low32(c>>32) = 0x274bec67
第一次运算	rol32(x,5) ^ 0xc44c = 0xe97d48a8
调用之间的异或	0xe97d48a8 ^ 0xe39d = 0xe97dab35
第二次运算	ror32(0xe97dab35,5) = 0xaf4bed59
最后加常量并输出	0xaf4bed59 + 0xa29d = 0xaf4c8ff6 = 2941030390

写入代码后，我执行 make 编译，再运行公开测试，10 个 seed 全部 PASS（图 12）。随后对 seed 43 做单关管道验证，输出 Phase 6 defused.（图 13）。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ make
python3 public_test.py --phase 6
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 12 第 6 关编译与公开测试命令、10/10 PASS 结果。

```
./defuser --seed 43 --phase 6 | ./bomb --seed 43 --phase 6
Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.
```

图 13 公开列表外 seed 43 的管道命令、challenge 与通过结果。

7 第 7 关（Honor）：观测台旧备份

第七行既要满足 secret_gate 对文本布局的要求，也要通过 secret_phase 对三个整数的检查。入口检查决定前缀和标记的位置，数值计算则同时用到 challenge 与 context。

7.1 确认入口、challenge 与 context 的来源

驱动的 --secret 模式先调用前六关求解函数，将六行答案交给 transcript_context 生成 64 位 context，再把第七关 challenge 与 context 传入 solve_secret。炸弹也在前六关通过后进入 Honor，检查第七行。

函数	从调用链确定的任务
secret_gate	检查整行长度、复制文本、检查固定标记，再调用 secret_phase
secret_copy	逐字节复制输入，包括末尾的 NUL 字节
secret_phase / secret_mix	解析三个整数，结合 challenge 与 context 计算期望值

7.2 第七行的长度、标记与偏移

```
40273b: lea  -0xd0(%rbp),%r12
40275c: call strlen
402761: sub  $0x24,%rax
402765: cmp  $0x7f,%rax
402769: jbe  402780
402786: call secret_copy
40278b: cmpl $0x4e45504f,-0xb0(%rbp)
40279d: lea  -0xac(%rbp),%rdi
4027a4: call secret_phase
```

令 L=strlen(line)。sub 后的无符号比较要求 L-36≤127；L<36 时减法下溢，比较也失败，所以长度范围为 36≤L≤163，不计末尾 NUL。长度恰为 36 时没有数字文本，仍会在后续解析失败。

复制起点是 rbp-0xd0，标记位置是 rbp-0xb0，两者相差 32 字节。常量 0x4e45504f 按小端顺序对应字节 4f、50、45、4e，即 OPEN。解析起点 rbp-0xac 比复制起点高 36 字节。

相对输入起点的字节偏移	应写入的内容
0-31	32 个 ASCII 字节；本实现选择 32 个 A
32-35	OPEN，恰好 4 字节
36 起	三个以空格分隔的无符号十进制整数

```
402710: movzbl (%rsi,%rax,1),%edx
402714: mov    %dl,(%rdi,%rax,1)
402717: add    $0x1,%rax
40271b: test   %dl,%dl
40271d: jne    402710
```

7.3 三个整数的计算

secret_phase 入口将 challenge 保存到 r12，将 context 保存到 r13。

字段检查及 sscanf 返回值均要求 3 项，格式串为 "%u %u %u"；三个输入分别位于 rbp-0x2c、rbp-0x28、rbp-0x24。令 c=challenge，k=context：

恢复的量	关键汇编	对应的计算或检查
a / 第一项	mov %r13d,%ecx and $3,%ecx cmp %ecx,-0x2c(%rbp)	a=low32(k)&3 第一项必须等于 a
A	mov %r12,%rsi shr $0x20,%rsi xor %r12d,%esi	A=low32(c>>32)^low32(c)
B	mov %r13,%r8 shr $0x20,%r8 xor %r13d,%r8d	B=low32(k>>32)^low32(k)
extra	imul $0x9493,%ecx,%r9d shr $0x10,%r13 xor %r13d,%r9d	extra=(a*0x9493)^low32(k>>16)
M(x)	mov %edi,%eax rol $5,%eax add $0x9493,%eax xor $0x4fe2,%eax	secret_mix 的返回值： M(x)=(rol32(x,5)+0x9493)^0x4fe2
第二项	mov %esi,%edi xor %r8d,%edi call secret_mix xor %r9d,%eax cmp %eax,-0x28(%rbp)	传入 M 的参数是 A^B second=M(A^B)^extra
第三项	shr $5,%esi lea 0x4fe2(%r8,%rcx,1),%edx add %esi,%edx xor %edx,%eax cmp -0x24(%rbp),%eax	esi 由 A 右移得到；eax 为 second third=second^(B+a+0x4fe2+(A>>5))

跟踪寄存器时，第二项使用 secret_mix 返回后再异或 extra 的结果；第三项继续使用这个 second。对照 secret_mix 的指令，调用前的 esi、r8d、ecx、r9d 在本段中仍保存 A、B、a、extra。

```
rol32(x, 5) = (x << 5) | (x >> 27);
first = a;
second = ((rol32(A ^ B, 5) + 0x9493u) ^ 0x4FE2u) ^ extra;
third = second ^ (B + a + 0x4FE2u + (A >> 5));
```

所有中间数和结果按 uint32_t 处理；加法、乘法和左移保留低 32 位。k>>16 必须先在原始 64 位 context 上执行，再取低 32 位；第三项中的 A>>5 则作用于已经折叠得到的 32 位 A。

7.4 在 solve_secret 中输出完整第七行

算出的 a、second、third 用 snprintf 写入 out。格式串前缀是 32 个 A 和 OPEN，PRIu32 用于输出三个 uint32_t 数。n<0 表示写入失败，n≥cap 表示空间不足，其余情况返回 0。

```
int n = snprintf(out, cap,
    "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN"
    "%" PRIu32 " %" PRIu32 " %" PRIu32,
    a, second, third);
return n < 0 || (size_t)n >= cap;
```

7.5 Honor 验证与完整运行

我重新编译后运行 python3 public_test.py --secret，10 个公开 seed 全部 PASS（图 14）。随后用 seed 43 运行完整七关，前六关和 Honor 均通过（图 15、图 16）。seed 43 的第七行按公式计算为：

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN3 3723420104 1289170538
```

这一行包含 32 个 A、4 字节 OPEN 和三个数字字段，总长 59 字节，满足入口的长度限制。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ make
python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

图 14 Honor 公开测试 10/10 通过。

```
wy123@WYLAPTOP11:~/ics-2026-bomblab$ {
    ./defuser --seed 43
    ./defuser --seed 43 --secret
} | ./bomb --seed 43
Phase 1 challenge: 0x9cde98852e60034b
Input (unsigned decimal integers):
Phase 1 defused.
```

图 15 seed 43 完整运行命令与第一关通过结果。

```
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。
Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

图 16 seed 43 完整运行结果：前六关及 Honor 均通过。

8 实验结果

第 1-6 关与 Honor 均通过各自的 10 个公开 seed。公开列表外的 seed 43 也完成了前六关及 Honor 的连续运行。

9 参考资料与辅助工具

[1] 课程实验说明与学生模板：求解接口、验证和提交要求。

[2] Linux sscanf(3) 手册：格式串、输出指针及成功赋值数量。

[3] GDB 官方文档与课程的 Linux x86-64 调用约定说明：断点、寄存器和内存查看。

工具：objdump 导出汇编；GDB 查看寄存器、数组、跳转表与节点数据；编辑器修改代码，GCC/Make 编译，脚本与管道测试。DeepSeek Harness、Codex 用于辅助解释汇编、整理求解代码和排版。我在 WSL 中编译运行，完成公开测试和 seed 43 的验证。